

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №6
По СПП

Выполнил:
студент 3 курса
группы ПО-12
Коханская Е.В.
Проверил:
Кулик А. Д.

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest

```
"""Tests for functions from Lab 1 (IntegerSet)."""
```

```
from typing import List
```

```
import pytest
```

```
class IntegerSet:
```

```
    """IntegerSet implementation from Lab 1."""
```

```
    def __init__(self, initial_elements: List[int] = None):
```

```
        self._elements: List[int] = []
```

```
        if initial_elements:
```

```
            for element in initial_elements:
```

```
                self.add(element)
```

```
    @property
```

```
    def elements(self) -> List[int]:
```

```
        """Return sorted list of elements."""
```

```
        return sorted(self._elements)
```

```
    @property
```

```
    def size(self) -> int:
```

```
        """Return number of elements."""
```

```
        return len(self._elements)
```

```
    def add(self, value: int) -> bool:
```

```
        """Add integer to set."""
```

```
        if not isinstance(value, int):
```

```
            raise TypeError(f'Only integers are allowed, got {type(value).__name__}')
```

```
if value not in self._elements:
    self._elements.append(value)
    return True
return False
```

```
def remove(self, value: int) -> bool:
    """Remove integer from set."""
    if not isinstance(value, int):
        raise TypeError(f"Only integers are allowed, got {type(value).__name__}")
    if value in self._elements:
        self._elements.remove(value)
        return True
    return False
```

```
def contains(self, value: int) -> bool:
    """Check if value exists in set."""
    if not isinstance(value, int):
        raise TypeError(f"Only integers are allowed, got {type(value).__name__}")
    return value in self._elements
```

```
def __len__(self) -> int:
    """Return number of elements."""
    return len(self._elements)
```

```
def __contains__(self, value: int) -> bool:
    """Check if value is in set using 'in' operator."""
    return self.contains(value)
```

```
class TestIntegerSet: # pylint: disable=too-many-public-methods
    """Test cases for IntegerSet class from Lab 1."""
```

```
# ===== TRIVIAL CASES =====
```

```
def test_empty_set_initialization(self):
```

```
    """Test creating empty IntegerSet."""
```

```
    int_set = IntegerSet()
```

```
    assert int_set.size == 0
```

```
    assert int_set.elements == []
```

```
def test_initialization_with_elements(self):
```

```
    """Test creating IntegerSet with initial elements."""
```

```
    int_set = IntegerSet([1, 3, 2])
```

```
    assert int_set.size == 3
```

```
    assert int_set.elements == [1, 2, 3]
```

```
def test_initialization_with_duplicates(self):
```

```
    """Test that duplicates are ignored during initialization."""
```

```
    int_set = IntegerSet([1, 2, 2, 3, 3, 3])
```

```
    assert int_set.size == 3
```

```
    assert int_set.elements == [1, 2, 3]
```

```
# ===== ADD METHOD TESTS =====
```

```
def test_add_new_element(self):
```

```
    """Test adding new element returns True."""
```

```
    int_set = IntegerSet()
```

```
    assert int_set.add(5) is True
```

```
    assert 5 in int_set
```

```
def test_add_existing_element(self):
```

```
"""Test adding existing element returns False."""
```

```
int_set = IntegerSet([1, 2, 3])
```

```
assert int_set.add(2) is False
```

```
assert int_set.size == 3
```

```
def test_add_multiple_elements(self):
```

```
    """Test adding multiple elements."""
```

```
    int_set = IntegerSet()
```

```
    for i in range(10):
```

```
        assert int_set.add(i) is True
```

```
    assert int_set.size == 10
```

```
# ===== REMOVE METHOD TESTS =====
```

```
def test_remove_existing_element(self):
```

```
    """Test removing existing element returns True."""
```

```
    int_set = IntegerSet([1, 2, 3])
```

```
    assert int_set.remove(2) is True
```

```
    assert 2 not in int_set
```

```
    assert int_set.size == 2
```

```
def test_remove_non_existent_element(self):
```

```
    """Test removing non-existent element returns False."""
```

```
    int_set = IntegerSet([1, 2, 3])
```

```
    assert int_set.remove(99) is False
```

```
    assert int_set.size == 3
```

```
def test_remove_until_empty(self):
```

```
    """Test removing all elements."""
```

```
    int_set = IntegerSet([1, 2, 3])
```

```
int_set.remove(1)
int_set.remove(2)
int_set.remove(3)
assert int_set.size == 0
assert int_set.elements == []
```

```
# ===== CONTAINS METHOD TESTS =====
```

```
def test_contains_existing_element(self):
    """Test contains returns True for existing element."""
    int_set = IntegerSet([10, 20, 30])
    assert int_set.contains(20) is True

def test_contains_non_existent_element(self):
    """Test contains returns False for non-existent element."""
    int_set = IntegerSet([10, 20, 30])
    assert int_set.contains(99) is False
```

```
def test_contains_with_in_operator(self):
    """Test 'in' operator works."""
    int_set = IntegerSet([1, 2, 3])
    assert 2 in int_set
    assert 99 not in int_set
```

```
# ===== LEN METHOD TESTS =====
```

```
def test_len_method(self):
    """Test __len__ method."""
    int_set = IntegerSet([1, 2, 3, 4, 5])
    assert len(int_set) == 5
```

```

def test_len_after_add(self):
    """Test length updates after add."""
    int_set = IntegerSet()
    assert len(int_set) == 0
    int_set.add(1)
    assert len(int_set) == 1

# ===== EDGE AND BOUNDARY CASES =====

def test_negative_numbers(self):
    """Test handling of negative numbers."""
    int_set = IntegerSet([-5, -3, -1])
    assert int_set.contains(-3) is True
    assert int_set.size == 3
    assert int_set.elements == [-5, -3, -1]

def test_zero_handling(self):
    """Test handling of zero."""
    int_set = IntegerSet()
    int_set.add(0)
    assert 0 in int_set
    assert int_set.size == 1

def test_large_numbers(self):
    """Test handling of large numbers."""
    int_set = IntegerSet([1000000, 10000000000, 10000000000000])
    assert int_set.size == 3

# ===== EXCEPTION CASES =====

```

```
def test_add_non_integer_raises_type_error(self):
    """Test adding non-integer raises TypeError."""
    int_set = IntegerSet()
    with pytest.raises(TypeError, match="Only integers are allowed"):
        int_set.add("string") # type: ignore
```

```
def test_remove_non_integer_raises_type_error(self):
    """Test removing non-integer raises TypeError."""
    int_set = IntegerSet([1, 2, 3])
    with pytest.raises(TypeError, match="Only integers are allowed"):
        int_set.remove("not an int") # type: ignore
```

```
def test_contains_non_integer_raises_type_error(self):
    """Test contains with non-integer raises TypeError."""
    int_set = IntegerSet([1, 2, 3])
    with pytest.raises(TypeError, match="Only integers are allowed"):
        int_set.contains(None) # type: ignore
```

```
# ===== PROPERTY TESTS =====
```

```
def test_elements_property_returns_sorted_copy(self):
    """Test that elements property returns sorted copy."""
    int_set = IntegerSet([3, 1, 4, 1, 5, 9, 2])
    elements = int_set.elements
    assert elements == [1, 2, 3, 4, 5, 9]
```

```
def test_elements_property_does_not_modify_internal(self):
    """Test that elements property returns copy, not reference."""
    int_set = IntegerSet([1, 2, 3])
```

```
elements = int_set.elements  
elements.append(99)  
assert int_set.elements == [1, 2, 3]
```